

# FINAL INGENIERIA DE SOFTWARE 1

**SOFTWARE**: Instrucciones, procedimientos, reglas, que forman parte de las operaciones de un sistema de computación.

Es lógico, se desarrolla, no se descarta

**SOFTWARE LIBRE**:

- Ejecutar el programa en cualquier sitio
- Estudiarlo y adaptarlo
- Redistribuirlo
- Mejorarlo

**CLASIFICACIÓN**: de sistemas, de app web, de IA ...

**INGENIERIA DE SOFTWARE**: Disciplina de la ingeniería que está dentro del marco económico, social y legal y comprende los aspectos de la producción de software durante todo su ciclo de vida. Tanto aspectos técnicos como gestión, herramientas

IEEE → métodos sistemáticos, disciplinados y cuantificables para el desarrollo, operación y mantenimiento

**PARTICIPANTES**:

- GERENTES EJECUTIVOS
- GERENTES DE PROYECTO
- PROFESIONALES
- CLIENTES
- USUARIOS FINALES

UN **INGENIERO DE SOFTWARE** debe dominar aspectos técnicos y habilidades blandas

Debe tener:

- CONFIDENCIALIDAD
- NO FALSIFICAR COMPETENCIAS
- CONOCER derechos de propiedad intelectual
- USO apropiado de computadoras

Lo que el usuario necesita → requerimientos

**REQUERIMIENTO:** característica del sistema o descripción de algo que hace el sistema para satisfacer su propósito.

### FUENTES DE REQUERIMIENTOS:

- Documentación
- Stakeholders: persona o grupo que se verá afectado por el sistema (usuarios, ingenieros, gerente...)
- Especificaciones de sistemas similares

### PUNTOS DE VISTA:

- De los interactuadores (personas o sistemas que interactúan)
- Indirecto (stakeholders que no interactúan ellos mismos pero pueden influir en los requerimientos)
- Dominio (características y restricciones del dominio)

**ELICITACIÓN DE REQUERIMIENTOS:** de carácter social principalmente. Adquirir todo el conocimiento necesario para producir un modelo de requerimientos

- Conocer el dominio
- Conocer el sistema actual
- Identificar las necesidades

{ Puede haber problemas de comunicación, cognitivos técnicos...

### TÉCNICAS DE ELICITACIÓN:

**Discretas** → Documentación existente  
→ Investigación y entrevistas al usuario.  
→ Observación al ambiente de trabajo

**INTERACTIVOS** → CUESTIONARIOS  
→ ENTREVISTAS  
→ PLANEACIÓN CONJUNTA DE REQUERIMIENTOS (JRP, JDA)  
→ BRAINSTORMING

Los discretos son menos perturbadores, más fáciles por sí solos

Documentación existente (organigramas, notas, rep. anteriores)

# Investigación y visitas al lugar

## Observación del ambiente de trabajo:

observar personas y actividades desde un bajo perfil sin interrumpir el trabajo pueden obtenerse datos confiables, distracciones físicas... puede incomodar

## INTERACTIVOS: más perturbadores

hablar y escuchar para comprender

- **QUESTIONARIOS**: documento que permite obtener info, sentimiento y opiniones generalizados mediante una serie de preguntas puede llegar a muchas personas. Permite cuantificar respuestas

✓ RESPUESTA RÁPIDA

✓ ANONIMOS

✓ ECONOMICOS

✓ ESTRUCTURADOS DE FÁCIL ANÁLISIS

X NUMERO BAJO DE RTAS

X NO RESPONDE A TODAS LAS PREG

X PREGUNTAS DÍGITAS

X NO ANÁLISIS CORPORAL

X DIFÍCIL DE PREPARAR +

Puede tener preguntas cerradas o abiertas  
(límites opuestos)

Abierta es de fácil preparación y difícil análisis

conclusiones, preparación, análisis, profundidad

¿Cuándo usar? Dispersos geográficamente  
muchas personas  
problemas generales

- **ENTREVISTAS**: técnica que recubre info a través de la interacción cara a cara mediante preguntas y respuestas opiniones sentimientos info

✓ ENTREVISTADO SE SIENTE INCLUIDO

✓ RETROALIMENTACIÓN

✓ LENGUAJE CORPORAL

✓ ADOPTACIÓN DE PREGUNTAS

X TIEMPO Y RECURSOS HUMANOS

X COSTOSA

X NO APLICABLE A CIERTAS

X ENTREVISTADOR ENTRENADO

Hoy entrevistas cerradas y abiertas

Cerradas: Sobre un requerimiento puntual conjunto específico de preguntas

Abiertas: Tema en general sin preparación de prep específicas

Las abiertas pueden dar detalles relevantes y perder el control

En la cerradas no se obtienen detalles. Si damos relevantes y control

## ORGANIZACIÓN:

INDUCTIVO (piramidal)  $\Delta$

DEDUCTIVO (embudo)  $\nabla$

DIAMANTE  $\diamond$

### PREPARACIÓN PREVIA

- \* Antecedentes
- \* Objetivos
- \* Entrevistados
- \* Planificar
- \* Tipo de preguntas

- PLANEACIÓN CONJUNTA DE REQUERIMIENTOS:  
reunion con grupo altamente estructurado con el fin de analizar problemas y definir requerimientos  
Ejercicio enmenamiento

✓ TIEMPO ahorrado  
✓ INNOVACIÓN nuevas  
✓ Creativo

X Organizar horarios  
X complejo encontrar  
participantes intérpretes

- BRAINSTORMING: técnica en la cual se generan tantas ideas como sea posible de ideas, en un periodo de tiempo corto sin posibilidad de análisis  
Promueve creatividad

✓ DESCUBRIR ideas  
✓ Ayuda en la falta de consenso  
✓ Encara dificultades del usuario de transmitir



# ESPECIFICACIÓN DE REQUERIMIENTOS :

Permite que los desarrolladores expliquen qué entienden sobre lo que el cliente espera

Indica a diseñadores funcionalidades

Indica al equipo de pruebas que demostrarles han

## PROPIEDADES DE LOS REQUERIMIENTOS :

- \* NECESARIO : su omisión provoca deficiencia
- \* CONUSO : fácil de leer y entender
- \* COMPLETO : No necesita ampliarse
- \* CONSISTENTE : No contradictorio
- \* NO AMBIGUO : una sola implementación
- \* VERIFICABLE : Puede testearse

## ASPECTOS BÁSICOS DE ESPECIFICACIÓN :

- \* FUNCIONALIDAD
- \* INTERFACES EXTERNAS
- \* RENDIMIENTO
- \* ATRIBUTOS
- \* RESTRICCIONES DE DISEÑO

## TÉCNICAS DE ESPECIFICACIÓN :

↳ **ESTÁTICAS** : se describe al sistema a través de las entidades u objetos, sus atributos y relaciones con otros. No describe cómo las relaciones cambian con el tiempo.

- \* Referencia indirecta
- \* Relaciones de recurrencia
- \* Definición axiomática
- \* Expresiones regulares
- \* Abstracción de datos

↳ **DINÁMICAS** : se considera al sistema en función de los cambios que ocurren con el tiempo. Un estímulo causa cambios de estado.

- \* Tablas de decisión
- \* Tablas de transición de estados
- \* Diagrama de transición de estados
- \* Redes de Petri

## HISTORIAS DE USUARIO: VER HOJA 7 ANTES

Descripción corta y simple de un requerimiento en un lenguaje común y desde la perspectiva del usuario Usadas en metodologías ágiles.

\* ESTIMABLES ENTRE 10HS y 2 SEMANAS (SINO DIVIDIR)

CARACTERÍSTICAS:

- \* PEQUEÑAS
- \* NEGOCIABLES entre clientes y desarrolladores
- \* ESTIMABLES
- \* INDEPENDIENTES UNAS DE OTRAS
- \* VERIFICABLES (en req. funcionales por lo que p mente son verificables)

### CRITERIOS DE ACEPTACIÓN:

Define si una HU fue desarrollada según la expectativa de los clientes y si se puede dar como hecha

Definida en etapa inicial complementa HU

Resúmenes de conversaciones entre cliente y desarrollador  
+ 4 → dividir

✓ RAPIDA IMPLEMENTACIÓN REQUISITOS  
✓ POCO MANTENIMIENTO  
✓ CERCANÍA CLIENTE  
✓ ESTIMABLES  
✓ PARA PROYECTOS CAMBIANTES

\* SIN CRITERIOS DE ACEPTACIÓN PUEDE QUEDAR ABIERTAS A DISTINTAS INTERPRETACIONES  
\* CONTACTO PERMANENTE  
\* DIFÍCIL ESCALAR PROYECTOS GRANDES  
\* DESARROLLADORES COMPETENTES

EPICA: conjunto de historias agrupadas por un denominador común  
OBJETIVO. NO ES FUNCIONALIDAD

# TIPOS DE REQUERIMIENTOS

↳ **FUNCIONALES**: describen interacción entre el sistema y su ambiente  
Indica cómo debe comportarse y como no debe  
Independiente de la implementación de la solución

↳ **NO FUNCIONALES**: describen restricción sobre el sistema que limita nuestras elecciones

↳ **DE PRODUCTO**: usabilidad, eficiencia, rendimiento, portabilidad

↳ **ORGANIZACIONALES**: políticas de la organización, entregas, implementación, estándares

↳ **EXTERNOS**: legales, privacidad, éticos

## INGENIERÍA DE REQUERIMIENTOS (specification)

Proceso por el cual se transforman los requerimientos en especificaciones precisas, no ambiguas, consistentes y completas <sup>del cliente</sup> del comportamiento del sistema  
↳ funcionalidades  
↳ base de acuerdos

Se interconectan distintos puntos de vista para modelar lo que el sistema realiza y se usa el documento de requerimientos.

<sup>iterativo</sup>  
Enfoque sistémico: re leer, organizar, documentar

IMPORTANCIA

- \* Permite gestionar necesidades estructurada
- \* Disminuye costos mente
- \* Mejora comunicación entre equipos
- \* Mejora calidad de Software

## ESTUDIO DE VIABILIDAD

Informe que recomienda la conveniencia o no de reanudar el proceso de desarrollo

Analizando riesgos, tecnología, costo-tiempo, integraciones

## VALIDACIÓN DE REQUERIMIENTOS

Centrarse en la corrección del modelo de req. contra intenciones us.  
Es importante porque los errores en los requerimientos pueden conducir a grandes costos

**VALIDACIÓN:** Al final del desarrollo, evaluar el software cumple con los requerimientos del usuario.  
Necesita la activa participación del usuario  
Hacer el software correcto

**VERIFICACIÓN:** el software cumple los requerimientos correctamente. Se mira el código y las pruebas

Hacer el software correctamente

|                |                                                                               |                                                                                                                                                                       |
|----------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VERIFICACIONES | * VALIDEZ<br>* CONSISTENCIA<br>* COMPLETUD<br>* REALISMO<br>* VERIFICABILIDAD | NO existen los<br>"requerimientos de<br>requerimientos"<br>NO puede probarse que es<br>correcto, puede alcanzarse<br>convicción de que es correcto para<br>el usuario |
|----------------|-------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|

Técnicas de validación pueden ser manuales o automáticas:

- Reuniones de requerimientos

- Informales: tratar con stakeholders
  - formales: con el cliente

- Prototipo

- Cases de prueba

¿ES suficiente validar después del desarrollo de sw?  
La evidencia estadística dice que NO  
Validar en especificación ✓

## CASOS DE USO:

Proceso de modelado de las "funcionalidades" del sistema en términos de los eventos que interacción entre los usuarios y el sistema



- BENEFICIOS:**
- \* CAPTURA REQUERIMIENTOS FUNCIONALES
  - \* DESCOMPONE el alcance del sistema en piezas más manejables
  - \* Permite estimar el alcance del proyecto y el esfuerzo
  - \* lenguaje común y fácil de entender
  - \* Únea base para planes de prueba y documentación

## COMPONENTES:

\* **DIAGRAMA DE CASOS DE USO**: ilustra interacciones entre el sistema y los actores

Compuesto por

↳ **CASOS DE USO**: Representa un objetivo (funcionalidad) individual del sistema. Debe estar acompañado de un escenario para que se verifique un rep.

▷ **ACTORES**: Inicia actividad (U) en el sistema. Representa un <sup>rol de un</sup> usuario que interactúa (rol) puede ser una persona, un sistema externo, un dispositivo externo

↳ **RELACIONES**:

- **ASOCIACIONES**: relación entre un actor y un cu en el que interactúan entre si
- **EXTENSIONES** (extend). Cu extiende funcionalidad a otro cu. solo son iniciados por un cu
- **USES**: Combina pasos comunes de los cu
- **Herencia**: relación entre actores donde un actor hereda las funcionalidades de otros

\* **ESCENARIOS** (narración del cu) es la descripción de la interacción entre el actor y el sistema para realizar la funcionalidad

## moduloso de cu:

Identificar autores

¿Quién proporciona entradas? ¿Quién recibe salidas?

Identificar los casos de uso

¿Tareas del autor? ¿Info?

CONSTRUIR EL DIAGRAMA

Realizar escenarios

## CARACTERÍSTICAS importantes:

- CU debe representar funcionalidad concreta
- En los pasos debe haber más de un paso (interacción)
- condiciones limitados a excepciones
- usos acudidos por lo menos entre 2 cu.

## DIAGRAMA DE TRANSICION DE ESTADOS (DTE)

Máquina de estado finito: describe al sistema como un conjunto de estados donde el sistema reacciona a ciertos eventos posibles (externos o internos)

Puede ser descrito como una 5-tupla

### COMPONENTES:

**EVENTOS:** suceso significativo que influye en el comportamiento del sistema

Tiene lugar en un punto de tiempo y con una duración temporal

evento [condición] / acción / es

**TRANSICION:** se producen como una secuencia de eventos

Pueden o no tener un procesamiento asociado

CONSTRUCCIÓN : \*

- \* Identificar estados
- \* Identificar estado inicial
- \* Analizar condiciones y acciones
- \* Verificar consistencia (que se puedan llegar a todos los estados y salir de todos los estados)

VENTAJAS/  
BENEFICIOS :

- \* MEJORA COMPRENSIÓN al ofrecer un mapa visual de como el sistema reacciona a ciertos eventos
- \* PREVIENE ERRORES lógicos
- \* Facilita las pruebas
- \* VALIDA El diseño. Confirma que todos los requisitos han sido considerados

¿Cuándo usar?

Sistemas de tiempo real  
Cuando el sw está impulsado por eventos externos  
Apps de interfaz de usuario complejas  
Sistemas con n° finito de estados  
Diseños de algoritmos de control

## REDES DE PETRI (RP)

Las redes de Petri especifican sistemas de tiempo real en los que es necesario representar aspectos de concurrencia

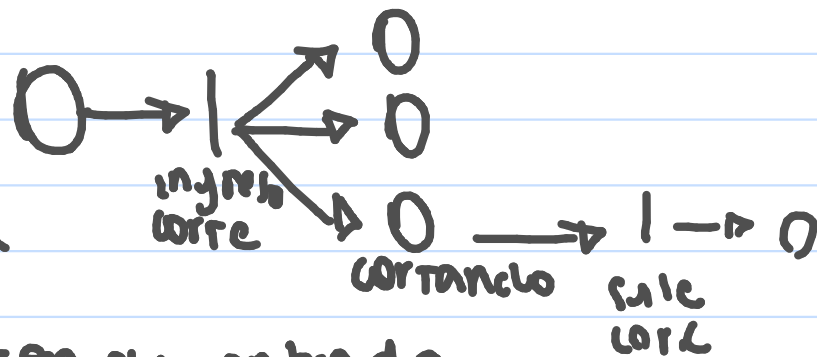
↳ (ejecución simultánea de tareas o procesos)

Las tareas concurrentes deben estar sincronizadas para permitir la comunicación entre ellas (debe prevenirse la modificación de datos compartidos o condiciones de bloqueo)

Pueden ejecutarse varias tareas en paralelo, en un orden

impredecible, no son deterministas

Los eventos se representan como transiciones (T)  
Los estados como sitios (P)



Una Red de Petri es una 4-tupla

$\{P \text{ lugares/sitios}, T \text{ transiciones}, I \text{ función de entrada}, O \text{ función de salida}\}$   
\* sitios modelan estados T (transición) modela eventos que cambian el estado

Multigrafo = de un nodo puede partir más de un arco

Bipartito = 2 tipos de nodos (T y P)

Dinámico = las flechas tienen dirección (los arcos) indica cómo se mueven los tokens

## CARACTERÍSTICAS:

- Los arcos indican con una flecha la relación entre sitios y transiciones.

- Después de la ejecución inicial se simula la ejecución número de token asignados a un sitio es limitado

- el conjunto de tokens sirven para manejar la concurrencia

- ocurre un evento y el token viaja

- El número y distribución de tokens controlan la ejecución → desactivar transiciones habilitadas

- \* arcos conectan transiciones con sitios  
o sitios con transiciones

y tokens que se mueven cuando la transición se dispara

- A los lugares/sitios se le asignan tokens (marcación)

- Limitada cuando cada lugar de entrada tiene el menor número de tokens como un caso



- La ocurrencia de los eventos (transiciones) depende del estado del sistema
- sujeto a ciertas condiciones  
y al ocurrir el evento se hacen verdaderas las post condiciones

SON ASINCRÓNICOS el orden de los eventos es uno de los permitidos \*  
hay q sincronizar p compartir datos y recursos

SON NO DETERMINÍSTICAS

\* ocurren  
c/ hay múltiples caminos no se puede predecir  
seguridad  
integridad

Beneficios:

- \* Permiten modular sistemas concurrentes
- \* Pueden ser aplicados a sistemas de control distribuido
- \* tienen formalidad matemática

asincrónico & no hay orden temporal fijo

## TABLAS DE DECISIÓN / ANÁLISIS ESTRUCTURADO

Herramienta que permite presentar de forma concisa las reglas lógicas a utilizar para decidir acciones a ejecutar en función de las condiciones y la lógica de un problema específico

SISTEMA como un conjunto de:

- Posibles condiciones
- Reglas para reaccionar cuando se reúnen determinado conjunto de condiciones
- Acciones a ser tomadas como resultado

CONSTRUIR SON :

- \* Condiciones y acciones simples

- \* Condiciones sólo V o F

- \* Hay  $2^n$  reglas donde  $n$  es la cantidad de condiciones

- ① Identificar condiciones y acciones

- ② Si hay condiciones opuestas colocar sólo una de ellas

- ③ construir reglas

ESPECIFICACIONES COMPLETAS : Determinan acciones para todas las reglas posibles

ESPECIFICACIONES REDUNDANTES : mismas condiciones acciones iguales

ESPECIFICACIONES CONTRADICTORIAS : mismas condiciones acciones distintas

## ANÁLISIS ESTRUCTURADO :

La elicitación proporciona una descripción verbal y una descripción visual ayuda a consolidar la info

Representación gráfica para comprensión más profundamente.

- Énfasis en procesamiento y transformación de datos

## DIAGRAMA DE FLUJO DE DATOS (DFD)

Herramienta que permite visualizar un sistema como una red de procesos funcionales, conectados por "conectores" y almacenamiento de datos,



# MODELOS DE PROCESO.

Proceso de desarrollo de Software:  
Conjunto de actividades que producen un producto de software

ACTIVIDADES PRINCIPALES de los procesos:

- ESPECIFICACION DE SOFTWARE (Unipenencia de requerimientos). Comprender y definir que requiere el sistema, junto con sus restricciones
- Desarrollo de SFW: convertir las especificaciones en un sistema ejecutable
- Validación del SFW: determinar si el sistema cumple con expectativas y especificaciones del usuario

## [ MODELO DE PROCESO DE SFW:

[ Es una representación simplificada de un proceso de software que presenta una visión de ese proceso  
Establece todas las actividades

Es un marco de referencia que contiene actividades y procesos involucrados en el desarrollo, evolución y mantenimiento desde inicio a fin

usa recursos. Sujeto a restricciones. genera productos intermedios y finales

Puede estar compuesto por subprocesos

Actividades organizados en secuencia

Es una representación abstracta

MODELOS - TIPOS:

- Prescriptivos
- Descriptivos
- Tradicionales
- Evolutivos



Prescriptivos : prescriben un conjunto de elementos del proceso : actividades, acciones, tareas, roles y prescribe un flujo de trabajo (circulación de elementos)

Descriptivos : descripción de la forma en que se realiza en la realidad

~

Deberían ser iguales

Tradicionales : Conjunto de fases o actividades en las que no se tiene en cuenta la naturaleza evolutiva del software

Clásico, lineal,  
Secuencial o en cascada, modelo en V  
basado en prototipos

Evolutivos : modelos que se adaptan a la evolución de los requisitos del sistema

En Espiral  
Evolutivo  
Incremental

## MODELO EN CASCAIDA :

modelo tradicional en el que las etapas se representan cayendo en cascada

Cada etapa debe completarse antes de que comience la siguiente

Dificultades : - No hay resultados concretos hasta que esté todo terminado

- los fallos más graves se encuentran al final y suele ser difícil solucionarlo

- "congelar" las fases es poco realista

- Existen errores, cambios de parecer

## MODELO EN V :

Modelo que demuestra como se relacionan las actividades de prueba con las de analisis y diseño

Sugiere que la prueba unitaria y de integracion tambien sea utilizada para definir el diseño del programa

La vinculacion entre lado izquierdo y derecho implica que si se encuentran problemas durante la Verificación y Validación entonces el lado izquierdo puede ser ejecutado nuevamente para solucionar el problema

## MODELO DE PROTOTIPOS :

Un prototipo es un producto parcialmente desarrollado y permite examinar aspectos del sistema y decidir si es adecuado para el producto terminado

Trata mejor la incertidumbre y ambigüedad

EVOLUTIVO : el objetivo es tener el sistema a entregar. Permite construcción rápidamente.

DESCARTABLE : sin funcionalidad. Con herramientas de modelado

PROYECTOS CANDIDATOS :

\* usuarios que no examinan modelos abstractos

\* usuarios que no determinan sus requerimientos inicialmente

A sistemas con énfasis en E/D

\* se enfatiza la interfaz humana

Éxito en sistemas en los que se pueda experimentar, barato, desarrollo rápidamente, mt humana, equipos reducidos

## MODELO DE DESARROLLO POR FASES :

Se desarrolla el sistema de manera que puede ser entregado por piezas. NO espera a terminar todo

Hay dos sistemas funcionando en paralelo

Operacional  
Sistema en desarrollo

Tipo incremental → cada entrega aporta un subsistema

Iterativo → entrega un sistema completo desde el principio y luego aumenta la funcionalidad de cada subsistema

En cascada es secuencial y rígido

## MODELO EN ESPIRAL (BOHEM)

Combina actividades de desarrollo con la gestión de riesgo

Incorpora objetivos de calidad ✓

Elimina errores desde el comienzo ✓

Permite iterar

Cada ciclo se completa con una reunión que incluye: todo el ciclo anterior y un plan para el siguiente

Cada ciclo tiene 4 sectores:

- Objetivos
- Valoración y reducción del riesgo
- Desarrollo y validación
- Plan de la siguiente fase

## METODOLOGÍAS ÁGILES

≠ controlado y riguroso

Es un enfoque iterativo incremental (evolutivo) del desarrollo de software

Prioridad a tareas que dan resultados directos y reducen burocracia tanto como sea posible

Enfoque en el cambio

## PRINCIPIOS:

- \* FÁCILES y CONTINUAS ENTREGAS DE SOFTWARE QUE FUNCIONA
- \* CAMBIOS DE REQUERIMIENTOS BIENVENIDOS
- \* USUARIOS y DESARROLLADORES TRABAJAN EN CONJUNTO
- \* CONSTRUYE PROYECTOS alrededor de personas, individuales
- \* Diálogo
- \* Atención continua a la excelencia técnica incrementando la agilidad
- \* Simplicidad

Org de equipos

## DESVENTAJAS

- \* A veces los clientes no intervienen por completo en el desarrollo de Software
- \* Priorizar cambios puede ser difícil
- \* La simplicidad requiere trabajo adicional y bajo presión de fechas de entrega puede perderse de tiempo
- \* Para algunas organizaciones resulta difícil moverse a procesos iterativos

minimiza la documentación

## PRINCIPALES METODOLOGÍAS ÁGILES:

EXTREME PROGRAMMING      SCRUM      KANBAN

EXTREME programming: valores de sencillez, colaboración, comunicación, retroalimentación, valentía y respeto

Prácticas simples

- ENFOQUE iterativo incremental (pequeñas mejoras)
- PRUEBAS unitarias continuas automatizadas (TESTING)
- Programación en pares
- Equipo integrado con cliente  
integración continua  
↳ presenté en el lugar de desarrollo
- Refactorización
- Simplicidad      Semana de 40 horas

## Roles:

**PROGRAMADOR** diseña, programa y realiza pruebas

**JEFE DE PROYECTO (manager)** reuniones. Asegura

**CLIENTE** determina que construya pruebas funcionales

**ENTRENADOR** responsable del proceso  
2º. Inno eflujo machine

**TESTER** aseguran que las pruebas funcionales se superen

**RASTRADOR** observa sin intervenir

## Proceso ...

## SCRUM

Marco de trabajo para desarrollar productos complejos aplicando buenas prácticas, trabajo colaborativo en equipos productivos

Equipos autoorganizados entregan resultados de calidad en tiempos cortos. Entrega frecuente, (iterativo incremental,



Sprint - "ciclo breve"

## PRINCIPIOS:

\* Eliminar el desperdicio

forar solapados.  
NO EN CONCORDIA

\* Construir la calidad con el producto

Desde el inicio con conocimiento. No se puede tener el conocimiento antes de empezar el desarrollo

\* Esperar el momento adecuado para la toma de decisiones

\* Entregar rápido

\* Respetar a las personas. Se trabaja mejor en ambiente que motive

\* Optimizar el todo

## ROLES

PRODUCT OWNER (Propietario)

↳ funcionalidad

Scrum master (jefe)

- Prioridades  
- asegurando reuniones

Scrum team (equipo)

- implementar funcionalidad

usuario o cliente

: bien finalizar para aportar idea

## ARTEFACTOS

PRODUCT BACKLOG

lista con prioridades deseadas

SPRINT BACKLOG

funcionalidad que el equipo se compromete a hacer en un sprint deter.m.

BURN DOWN

cuando usar?

Proyectos con caos constante

Req. cambiantes

Tecnología de punta

## KANBAN "Tarjeta visual"

3 practicas: limitar el trabajo (unos pocos)  
visualizar el flujo de trabajo  
medir el tiempo de entrega

No define rol ni secuencia de pasos

- Tareas realizadas
- A realizar
- Pendientes

Tablero con columnas que representan estados de repeticiones

## DESARROLLO DE SOFTWARE BASADO EN MODELOS:

Un modelo de software es una conceptualización del dominio del problema y ~~actua~~ como una especificación de los requerimientos

### MODEL DRIVEN DEVELOPMENT (MDD)

Asigna a modelos rol central y activo

Promueve

- \* abstracción en especificación
- \* Aumento de confianza en automatización

- \* Estándares industriales como medio para facilitar comunicación

- \* Modelos conductores primarios de los aspectos de desarrollo de software

entidades productivas

Platform independent model (Pin)  
Platform specific model (Ps)

Beneficios MDD:

- \* Incremento de productividad
- \* Adaptación a cambios tecnológicos y de requisitos
- \* Consistencia
- \* Re-uso
- \* Mejora comunicación
- \* Copiando experiencias
- \* Modelos son productores de mayor claridad
- \* Posibilidad de demorar decisiones tecnológicas

## CALIDAD

Según las principales normas internacionales:

(ISO 9000) Grado en que un conjunto de características inherentes cumplen sus requisitos

(ISO 8402) conjunto de propiedades o características..

Sistemas de información: Personas, datos, procesos, tecnologías... dar soporte a organización  
No sólo computacional

Calidad del SW → del producto  
↳ del proceso

ISO 25010 características:

PORTABILIDAD

SEGURIDAD

FACILIDAD DE MANTENIMIENTO

FACILIDAD DE USO

EFICIENCIA

CMM - CMMI - modelo de evaluación de los procesos de una organización

CMMI

madurez de la organización

↳ Escalonado

↳ Continuo

→ en el nivel de capacidad  
que tan bien se cumple  
la organización

ISO Normas de gestión de calidad  
aplicables a cualquier tipo de organización  
obtener mejoras y certificación

\$\$

legal

satisfacción

de clientes

} beneficios

Calidad Producto - 150/1EC  
25000  
Calidad Proceso - 150/1EC  
CMMI  
Calidad Productory  
Servicios en general 150 7001

## ÍNDICE

1. SOFTWARE
2. REQUERIMIENTOS
5. ESPECIFICACIÓN DE REQUERIMIENTOS
6. HU
8. CU
10. DTE
11. RP
13. TABLAS de decisión
16. Modelos de Proceso
19. Metodologías ágiles
24. Calidad